

Práctica EJEMPLO - React.js - IWEB

Importante, esta práctica no cuenta para nota al ser una práctica de ejemplo para hacer en clase. Con ella aprenderemos a seguir los pasos solicitados y a utilizar la herramienta “autocorrector”

Introducción y Objetivos

Requisitos:

- Nociones básicas de las tecnologías explicadas en clase (HTML, CSS, JS, **React**)
- Editor de código Visual Studio Code (o similar)
- Tener instalado [node.js](#) **versión 22 o 24** (LTS) y [git](#)

Motivación:

Intenta mirar esta práctica de ejemplo como el modo de arrancar con la asignatura, ver cómo se hacen pruebas manuales y cómo se hacen pruebas con el autocorrector.

Nos han pedido hacer esta aplicación con estas características mínimas y estos requisitos. Y para que nos paguen (y por lo tanto obtener el 10/10) debe cumplir con estas especificaciones y para ello nos pasan una batería de tests que debe pasar.

Objetivo:

El objetivo de esta práctica es realizar una Single Page Application (SPA) de un buscador de usuarios. Esta aplicación al cargar realiza una query a un servidor (API) y mostrará los resultados. También tiene un botón con el que traerse todos los usuarios.

Los usuarios se presentan en tarjetas como muestra la siguiente captura.

 Bienvenido a la página de Enrique Barra

Buscador de usuarios

Texto a buscar

Buscar Ver Todos

Nombre: Terry Medhurst Email: atuny0@sohu.com 	Nombre: Sheldon Quigley Email: hbingley1@plala.or.jp 	Nombre: Terrill Hills Email: rshawe2@51.la 
Nombre: Miles Cummerata Email: yraigatt3@nature.com 	Nombre: Mavis Schultz Email: kmeus4@upenn.edu 	Nombre: Alison Reichert Email: jtreleven5@nhs.uk 

Página principal

Primeros pasos: Preparación del entorno de desarrollo

Para poder hacer esta aplicación se proporciona un esqueleto o scaffold en github con el resultado de ejecutar el comando “yarn create vite” y añadir la herramienta de autocorrección y los tests a pasar para superar esta práctica.

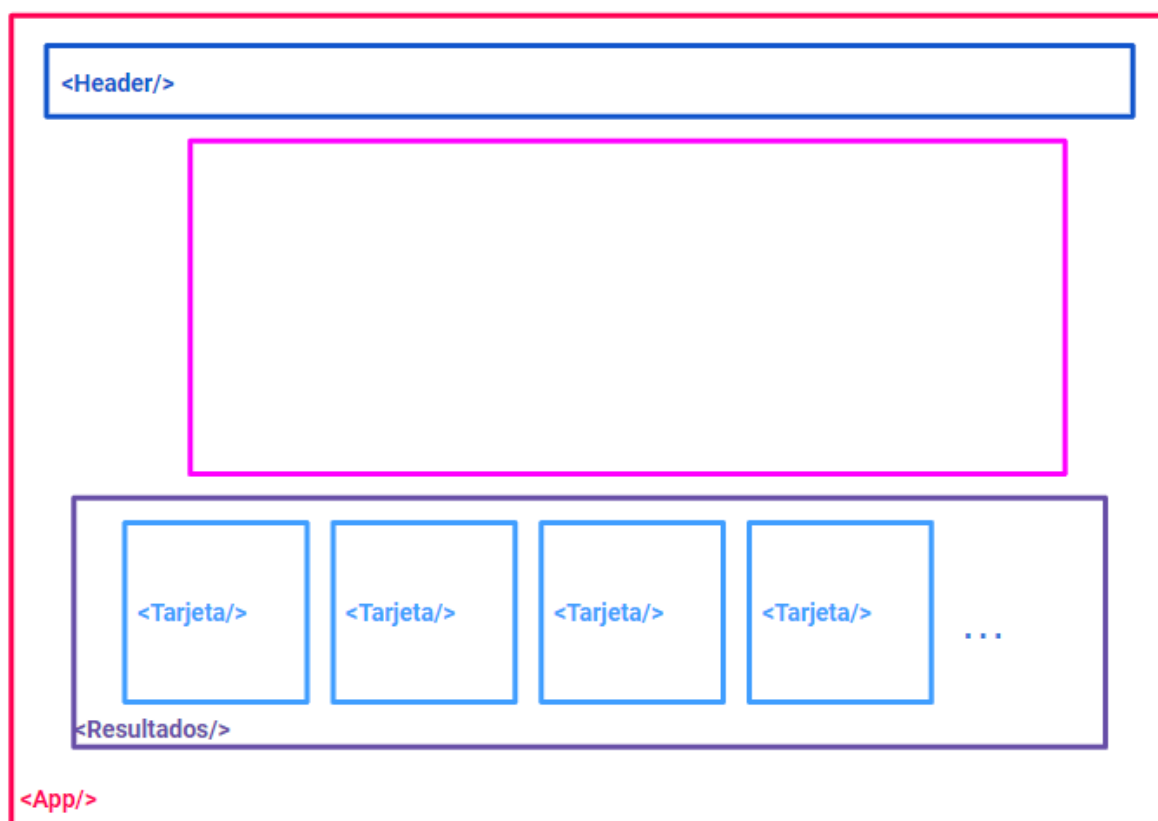
1. Clona el esqueleto de la aplicación a tu ordenador: `git clone https://github.com/REACT-UPM/ejemplo_buscador_usuarios.git`
2. Entra en el directorio creado: `cd ejemplo_buscador_usuarios`
3. Instala los paquetes necesarios que indica el package.json: `yarn`
4. Inicia el servidor de desarrollo: `yarn run dev`

A partir de ahora, si abrimos el navegador en la URL <http://localhost:5173>, podemos visualizar la aplicación en tiempo real mientras desarrollamos. Cuando queramos parar el desarrollo basta con hacer `Control + C` en el terminal.

Ahora hay que abrir la carpeta “ejemplo_buscador_usuarios” en el editor Visual Studio Code y empezar a desarrollar nuestra aplicación.

Comenzando a desarrollar

Antes de empezar a programar, uno debe pensar cómo estructurar la aplicación, y decidir *grosso modo* qué componentes va a necesitar y la jerarquía de los mismos. En esta práctica los componentes App, Header y Resultados se tienen que llamar así por definición.



También se recomienda definir el estado de nuestra aplicación (aunque sea una primera aproximación que luego irá mutando según añadamos funcionalidades). ¿Qué **información** necesito para pintar esta aplicación?

API de usuarios

En esta aplicación utilizaremos un API disponible en DummyJson. En este caso no es un api real sino un API Rest para practicar, pero está bien documentada y es más o menos potente.

La documentación del API está en:

<https://dummyjson.com/docs/users>

El endpoint es:

<https://dummyjson.com/users>

Como en la P1 y P2 a la hora de desarrollar aplicaciones que “atacan” APIs hay que utilizar unos datos fake para ir haciendo pruebas con datos controlados y luego cuando tengamos todo desarrollado implementamos la llamada al API y refinamos lo que falte. Para este propósito se provee en el proyecto de github en la carpeta *src/constants* un fichero js que exporta un mock de los datos. Examine dicho fichero para ver el formato de los datos.

Configuración de la aplicación (fichero config.js)

Para esta aplicación se provee un fichero en el proyecto de github en la carpeta *src/config* llamado *config.js* que exporta una configuración inicial propuesta. Examinar dicho fichero para ver las variables que exporta.

Requisitos que debe cumplir nuestra aplicación

La práctica se valorará sobre un máximo de 10 puntos. La mayoría de los requisitos son de funcionalidad, para la interfaz de usuario puede maquetar como desee.

Requisitos:

- **2 punto:** La aplicación tiene un componente Header con el logo y el mensaje de bienvenida con tu nombre (el nombre debe coincidir con el que le hemos dado al autocorrector la primera vez que lo ejecutamos).
 - El componente Header debe renderizar un elemento div con el id “cabecera”
 - Dentro de dicho div debe tener una imagen con la clase “logo” y un h3 con la clase “mensaje”
 - El h3 debe contener el texto que desee, pero al menos debe poner tu nombre. Por ejemplo, como se ve en la captura inicial de esta práctica “Bienvenido a la página de Enrique Barra”
- **2 punto:** La aplicación tiene al menos un h2, un input y un button
 - El h2 debe tener el id “buscador”
 - El input debe tener el id “query” y el button debe tener el id “botonsearch”
- **2 punto:** La aplicación tiene un componente Resultados que renderiza los productos que recibe
 - El componente Resultados tiene que recibir una prop “resultado” con un array con los usuarios (inicialmente mockdata.users y posteriormente vendrá de una query al servidor)
 - IMPORTANTE: Se debe utilizar un ul con un id “resultados” y cada usuario debe ser un . Esto será lo que use el corrector para buscar los resultados y contarlos y comprobar las funcionalidades.
- **2 punto:** La aplicación hace fetch de datos del servidor remoto al hacer click en el botón “ver todos”
 - cuando el parámetro use_server vale true en el fichero de config se hace la petición al servidor y se utilizan los datos que devuelve
 - la aplicación tiene un botón “ver todos” con id “botonall”

- la url del servidor se obtiene también del fichero de configuración del parámetro `server_url`
- **2 punto:** La aplicación maneja el valor del input (componente controlado de React) y filtra los resultados por su título al pulsar el button
 - No realizar el filtrado localmente sino en el servidor, es decir realizar una query con el parámetro search <https://dummyjson.com/docs/users>

Pruebas manuales y capturas de pantalla

Como se ha comentado se provee una batería de tests que la aplicación debe pasar y que serán los que produzcan la nota final. Pero es importante desarrollar la aplicación viendo poco a poco el resultado y visualizando cada cambio introducido hasta que funcione. Considerando los test provistos por el autocorrector como una serie de requisitos “estrictos” o “estáticos” como por ejemplo que tenga determinado id o clase alguna etiqueta o que el componente se llame de determinada manera.

No se recomienda por lo tanto ir desarrollando sin visualizar lo que hacemos y pasar el autocorrector a cada paso “a ver si se consiguen los puntos”, porque los errores que dan las baterías de tests son más crípticos que lo que veremos en el navegador o en la consola por nosotros mismos al visitar <http://localhost:3000>

Adicionalmente a pasar la batería de tests y obtener un 10/10 hay que hacer dos capturas de pantalla con la aplicación completa, es decir en la versión final antes de entregar. Dichas capturas se tienen que colocar en formato png, jpg o pdf en el directorio “miscapturas”. Y el autocorrector las subirá junto con el código de la práctica y el resto de evidencias a Moodle. Estas capturas son obligatorias y deben ser personales, en ellas se debe ver que la cabecera de la práctica pone el nombre del alumno y que el estilo es el entregado en el código. Estas capturas serán revisadas por el profesor para comprobar que se ha realizado la funcionalidad correctamente.

En esta práctica hay que subir una captura similar a la siguiente

 Bienvenido a la página de Enrique Barra

Buscador de usuarios

Nombre: **Terry** Medhurst
Email: atuny0@sohu.com



Nombre: **Sheldon** Quigley
Email: hbingley1@plala.or.jp



Nombre: **Terrill** Hills
Email: rshawe2@51.la



Nombre: **Miles** Cummerata
Email: yraigatt3@nature.com



Nombre: **Mavis** Schultz
Email: kmeus4@upenn.edu



Nombre: **Alison** Reichert
Email: jtreleven5@nhs.uk



Pruebas con el autocorrector

El autocorrector es la herramienta que permite pasar la batería de tests a la práctica y producir una nota. También subirla a Moodle junto con el código desarrollado, las capturas y otras evidencias de evaluación.

Ejecute el autocorrector tantas veces como desee en la práctica y suba la nota a Moodle también tantas veces como desee hasta que se cierre la entrega.

Dudas y tutorías

Para dudas sobre el enunciado y sobre la práctica en general vamos a utilizar el foro de la asignatura. Lo único que está prohibido es subir vuestra solución completa al foro en un zip o enlace o similar. Si se pueden compartir trozos de código para pedir ayuda o para ilustrar lo que os está ocurriendo.

Como insistimos en la guía de la asignatura y comentamos el día de la presentación la copia de la práctica es un suspenso automático de toda la asignatura (y posibilidad de tomar medidas disciplinarias a nivel UPM dependiendo de la gravedad del caso) tanto para el que copia como para el que se deja copiar. Así que por favor se prudente y no compartas tu código como un zip con tus compañeros, una vez que sale de tu ordenador no sabes quien puede entregar tu código en su nombre.

Contacto (aunque insisto que para las dudas es mejor vía foro):
Enrique Barra Arias - enrique.barra@upm.es